Neural Networks & Backpropagation

From the Perceptron to Deep Learning

Lebnet Tech Fellows Program

Roadmap

The Perceptron

The Limits of Linearity

Universal Approximation

Training: Loss and Optimisation

Backpropagation

Going Deep: Challenges

The Big Picture

A Neuron-Inspired Classifier

Given labelled data $(x, l(x))$ with $x \in \mathbb{R}^n$, $l(x) \in \{+1, -1\}$:

$$l(x) = \text{sign}(\langle w^*, x \rangle)$$

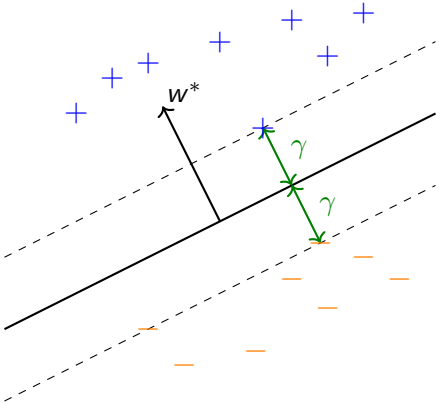
w^* is the normal vector to a separating hyperplane.

Perceptron Algorithm

1. Initialise $w = 0$
2. For each input x: predict $\text{sign}(\langle w, x \rangle)$; on a mistake, update $w \leftarrow w + l(x)\, x$

Each update **tilts the hyperplane** toward the misclassified point.

Visualising the Margin



The margin γ is the minimum distance from any point to the decision boundary.

Larger margin $\Rightarrow$ fewer mistakes.

Convergence Guarantee

Definition (Margin)

$\gamma := \min_{x \in D} |\langle w^*, x \rangle| \quad$ (assuming $\|w^*\| = 1$, $\|x\| \leq 1$)

Perceptron Convergence Theorem

The number of mistakes is at most $\dfrac{1}{\gamma^2}$.

Proof idea: track $\cos(w, w^*)$. Each mistake grows the numerator by $\geq \gamma$ but the denominator by $\leq \sqrt{T}$. Cauchy-Schwarz forces $T \leq 1/\gamma^2$.

Linear Models Cannot Be Stacked

Key Fact

A composition of linear (affine) functions is still linear:

$$(g \circ f)(x) = (CA)x + (Cb + d)$$

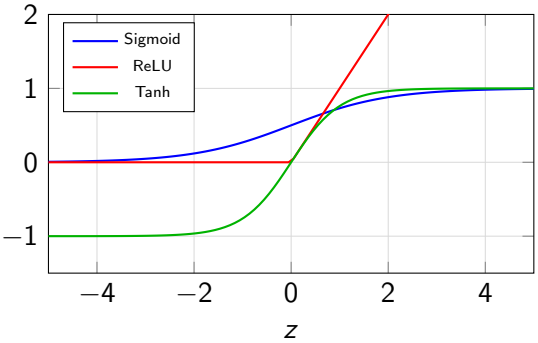
A 100-layer linear network = a single-layer network.

To learn nonlinear decision boundaries we need **activation functions** applied element-wise:

$$h^{(l)} = \sigma\big(W^{(l)} h^{(l-1)} + b^{(l)}\big)$$

Common Activation Functions

Name	**Definition**
Sigmoid	$\frac{1}{1+e^{-z}}$
Tanh	$\frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$
ReLU	$\max(0, z)$
Leaky ReLU	$\max(\alpha z, z)$



Neural Nets Can Approximate Anything

Universal Approximation Theorem

For any continuous $f : [0,1]^d \to \mathbb{R}$ and any $\varepsilon > 0$, there exists a single-hidden-layer network

$$g(x) = \sum_{j=1}^{N} c_j \, \sigma(\langle w_j, x \rangle + b_j)$$

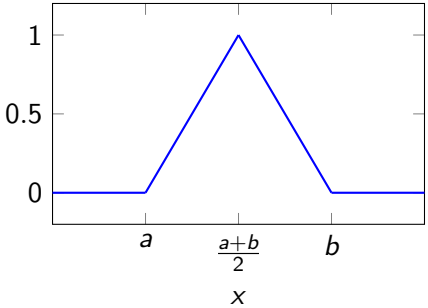
such that $\sup_x |f(x) - g(x)| < \varepsilon$.

Holds for any continuous, non-polynomial activation (sigmoid, tanh, ReLU, ...).

Intuition: ReLU Bump Functions

Two ReLUs make a bump; scaled bumps approximate any curve.

Single bump

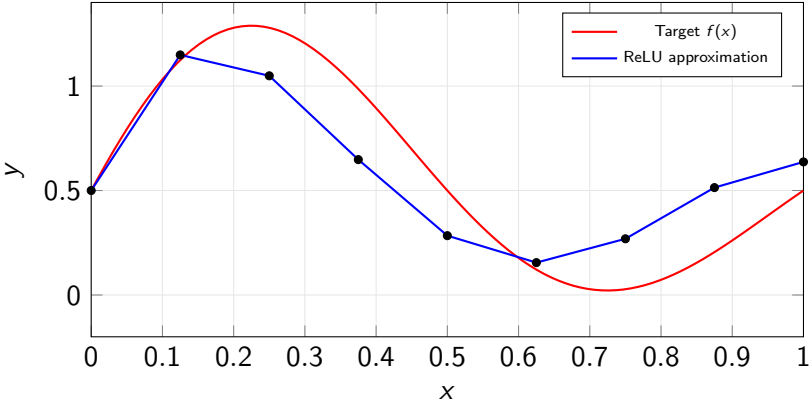


- Partition $[0, 1]$ into n intervals
- Place a scaled bump on each interval
- Piecewise linear $\rightarrow$ arbitrary precision by uniform continuity

A network with n hidden ReLU units can represent any piecewise linear function with n pieces.

Example: Approximating a Function with ReLU Bumps

Approximating $f(x) = \sin(2\pi x)\, e^{-x} + 0.5$



The Training Objective

Network $f_\theta : \mathbb{R}^d \to \mathbb{R}^k$ with weights θ, training data $\{(x_i, y_i)\}_{i=1}^n$.

$$\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell\big(f_\theta(x_i),\ y_i\big)$$

Task	**Loss**	**Formula**
Regression	MSE	$\frac{1}{2}\|\hat{y} - y\|^2$
Binary classif.	Cross-entropy	$-y \log \hat{y} - (1-y)\log(1-\hat{y})$
Multi-class	Softmax CE	$-\log(\mathrm{softmax}(\hat{y})_y)$

SGD: Scalable Optimisation

Full gradient descent: $\theta_{t+1} = \theta_t - \eta \nabla \mathcal{L}(\theta_t)$ (cost $O(n)$ per step)

Stochastic gradient descent: sample one (or a mini-batch of) data point(s):

$$\theta_{t+1} = \theta_t - \eta_t \nabla \ell\big(f_{\theta_t}(x_i),\, y_i\big)$$

- **Unbiased:** $\mathbb{E}_i[\nabla \ell_i] = \nabla \mathcal{L}(\theta)$
- Cost $O(1)$ per step (vs. $O(n)$)
- Converges with decaying learning rate

The Gradient Problem

A neural network is a **computational graph** (DAG).

Goal: compute $\frac{\partial \mathcal{L}}{\partial w}$ for every weight w.

By the chain rule at node v with downstream children $w_1, \ldots, w_m$:

$$\frac{\partial f}{\partial v} = \sum_{j=1}^{m} \frac{\partial f}{\partial w_j} \cdot \frac{\partial w_j}{\partial v}$$

Forward mode: propagate $\frac{\partial v}{\partial \theta}$ forward for *each* weight. Cost: $O(WE)$.
Problem: terms like $\frac{\partial f}{\partial v}$ are recomputed from scratch for every weight.

Backward Mode: Share the Work

Key insight: compute $\frac{\partial \mathcal{L}}{\partial v}$ for *all* nodes in a single backward sweep.

Backpropagation

Forward pass: compute and store all activations $v^{(l)}$.
Backward pass (from output to input):

1. $\delta^{(L)} = \frac{\partial \mathcal{L}}{\partial v^{(L)}}$
2. For $l = L-1$ down to 1:
   - $\delta^{(l)} = (W^{(l+1)})^\top \delta^{(l+1)} \odot \sigma'(u^{(l)})$
   - $\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l)} (v^{(l-1)})^\top$

Why Backprop Wins

Each edge is visited **once** forward, **once** backward.

Method	**Cost**
Numerical (finite diff.)	$O(WE)$
Forward-mode autodiff	$O(WE)$
Backpropagation	$\boldsymbol{O(E)}$

W = number of weights, E = number of edges.
Backprop is what makes training large networks feasible.

Vanishing Gradients

With L sigmoid layers, the gradient at layer l involves:

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} \propto \prod_{k=l}^{L} \sigma'(u^{(k)}) \cdot W^{(k)}$$

Since $\sigma'(z) = \sigma(z)(1 - \sigma(z)) \leq \frac{1}{4}$ for all z:

$$\left\| \frac{\partial \mathcal{L}}{\partial W^{(1)}} \right\| = O\left(\frac{1}{4^{L-1}} \right)$$

Gradients **shrink exponentially** with depth. Early layers barely learn.

Exploding Gradients

If $\|W^{(k)}\| > 4$, the product grows exponentially instead:

$$\left\| \frac{\partial \mathcal{L}}{\partial W^{(1)}} \right\| = O(\lambda^{L-1}), \quad \lambda > 1$$

Weights oscillate wildly; training diverges.

The core tension

Sigmoid derivatives are $\leq 1/4$. Multiply enough of them $\Rightarrow$ vanish. Scale weights up to compensate $\Rightarrow$ explode.

Modern Solutions

1. **ReLU activation:** $\sigma'(z) \in \{0, 1\}$, no shrinking in the active region.

2. **Careful initialisation:**
   - Xavier (Glorot): $\mathrm{Var}(w) = \frac{2}{n_{\text{in}} + n_{\text{out}}}$ (tanh/sigmoid)
   - He: $\mathrm{Var}(w) = \frac{2}{n_{\text{in}}}$ (ReLU)

3. **Batch normalisation:** normalise activations per mini-batch to $\hat{z} = \frac{z - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$.

4. **Residual connections (ResNets):** $h^{(l+1)} = h^{(l)} + F(h^{(l)})$. Gradients flow through the identity path.

The Progression of Ideas

Model	**Key Insight**	**Limitation**
Perceptron	Linear separation	Only linearly separable data
SVM	Max margin + kernels	Fixed feature space
Neural Net	Nonlinear activations $\rightarrow$ UAT	Training is non-convex
Deep Nets	Depth gives expressiveness	Vanishing/exploding gradients

Key takeaways

1. Nonlinearity is **essential**: without it, depth is useless.
2. One hidden layer **suffices** in theory (UAT), but depth helps in practice.
3. Backprop computes all gradients in $O(E)$, making large-scale training possible.
4. ReLU, good init, and residual connections tame the gradient flow.

Questions?